

Universidad Tecnológica Nacional

Facultad Regional San Francisco

Tecnicatura Universitaria en Programación

Trabajo Final Integrador

MarketSaaS

*“Plataforma Multi-inquilino para la Gestión y
Comercialización de Emprendimientos”*

DESARROLLADO POR

Bessone Mariano

Legajo: 17274

DOCENTE TUTOR

Cardona, Fernando

CICLO LECTIVO

2026

SAN FRANCISCO, CÓRDOBA

REPÚBLICA ARGENTINA

ÍNDICE

2. RESUMEN.....	2
2.1 Resumen Ejecutivo.....	2
2.2 Contexto del problema y motivación.....	2
3. OBJETIVOS DEL PROYECTO.....	2
3.1 Objetivo General.....	2
3.2 Objetivos Específicos.....	3
4. ANÁLISIS DEL PROBLEMA Y REQUISITOS.....	4
4.1 Descripción del problema real o necesidad detectada.....	4
4.2 Alcance y limitaciones del sistema.....	4
Alcance:.....	4
Limitaciones:.....	5
4.3 Principales requerimientos.....	5
Funcionales:.....	5
No Funcionales:.....	5
5. DISEÑO DE SOFTWARE.....	6
5.1 Arquitectura general.....	6
5.2 Tecnologías a utilizar.....	6
5.2.1 Backend.....	6
5.2.2 FrontEnd.....	7
5.2.3 Gestor de Base de datos.....	7
5.2.4 Frameworks.....	7
6. PRUEBAS Y CALIDAD DE SOFTWARE.....	7
6.1 Estrategia y Niveles de Prueba.....	8
6.2 Tabla de Casos de Prueba por Rol de Usuario.....	8
6.3 Pruebas de Calidad No Funcionales.....	9
6.4 Estándares de Calidad y Mantenibilidad.....	10
7. CRONOGRAMA DEL PROYECTO.....	10
7.1 Planificación temporal (Diagrama de Gantt).....	10
7.2 Detalle de Etapas Sugeridas.....	11

2.RESUMEN

2.1 Resumen Ejecutivo

El presente proyecto final aborda el desarrollo de una plataforma **SaaS Multi-tenant** escalable, diseñada para centralizar la comercialización digital de diversos emprendimientos en una infraestructura única. Mediante el uso de **.NET 8** y **Vue.js**, el sistema provee a cada inquilino un marketplace autogestionable con catálogo dinámico, control de stock transaccional y un **módulo de mensajería síncrona vía SignalR** para la atención al cliente. La seguridad se garantiza mediante **Tokens JWT** para el aislamiento de datos, mientras que la persistencia en **MongoDB** ofrece flexibilidad para diversos rubros comerciales. La solución integra la **API de Mercado Pago** para automatizar cobros y herramientas de analítica, optimizando la toma de decisiones y reduciendo costos operativos globales.

2.2 Contexto del problema y motivación

En el escenario actual, los pequeños emprendimientos enfrentan una brecha tecnológica crítica. La gestión de ventas a través de canales informales deriva en la pérdida de trazabilidad, falta de control de inventario y una experiencia de usuario deficiente. La motivación de este proyecto es aplicar conceptos avanzados de **Ingeniería de Software** para proveer una solución profesional que permita la transformación digital masiva. El desafío técnico de garantizar el **aislamiento de datos** entre negocios y la sincronización en tiempo real de stock y mensajería es el motor principal de esta investigación.

3. OBJETIVOS DEL PROYECTO

3.1 Objetivo General

Diseñar y desarrollar una plataforma web de tipo **Software as a Service (SaaS)** con arquitectura multi-inquilino que permita a múltiples negocios gestionar de forma segura e independiente su ciclo de ventas, stock y atención al cliente.

3.2 Objetivos Específicos

Para alcanzar el objetivo general, se plantean los siguientes hitos técnicos y funcionales:

- **Implementar un esquema de Autenticación y Autorización mediante JSON Web Tokens (JWT):** Desarrollar una capa de seguridad *stateless* que permita gestionar sesiones seguras, persistentes y basadas en roles (SuperAdmin, Admin de Negocio, Cliente), garantizando la integridad de los accesos a la API.
- **Desarrollar una Arquitectura Multi-tenant mediante Identificadores de Contexto:** Diseñar la lógica de backend para resolver rutas dinámicas y asegurar el aislamiento estricto de la información, permitiendo la coexistencia de múltiples unidades de negocio bajo una infraestructura de código única.
- **Diseñar un Motor de Control de Stock Transaccional:** Programar una lógica de negocio que valide y actualice existencias en tiempo real, vinculada a eventos de pago confirmados para evitar inconsistencias de inventario y sobreventas.
- **Establecer una Persistencia Documental Flexibilizada en MongoDB:** Modelar una base de datos NoSQL que permita la definición de atributos variables en los catálogos de productos, adaptándose a las necesidades específicas de diversos rubros comerciales sin requerir migraciones de esquema.
- **Integrar Comunicación Bidireccional mediante WebSockets (SignalR):** Implementar un módulo de mensajería síncrona para ofrecer soporte técnico y comercial en tiempo real, optimizando la interacción entre el cliente y el administrador del negocio.
- **Automatizar la Gestión de Cobros mediante el SDK de Mercado Pago:** Integrar la pasarela de pagos para el procesamiento de transacciones y el manejo asíncrono de estados de pedidos mediante Webhooks.
- **Desarrollar un Módulo de Analítica de Datos (Business Intelligence):** Programar un dashboard de reportería que procese datos históricos de ventas para identificar tendencias de consumo, productos de alta rotación y KPIs comerciales por cada inquilino.
- **Configurar el Despliegue en Entornos Cloud (Hosting y Deployment):** Realizar la puesta en producción de la solución en un servidor en la nube, asegurando la alta disponibilidad, la estabilidad del sistema y el acceso público bajo condiciones de tráfico real.
- **Optimizar la Experiencia de Usuario (UX) mediante Arquitectura SPA:** Utilizar las capacidades reactivas de **Vue.js** para construir una interfaz fluida que minimice la latencia de carga y ofrezca una navegación dinámica e intuitiva.
- **Implementar un Plan de Pruebas de Software (QA) y Validaciones de Datos:** Desarrollar mecanismos de validación tanto en el *Client-side* como en el *Server-side* para asegurar la integridad de la información ingresada, realizando pruebas de integración que verifiquen el correcto flujo de datos entre los distintos módulos del sistema.

4. ANÁLISIS DEL PROBLEMA Y REQUISITOS

4.1 Descripción del problema real o necesidad detectada

En el actual ecosistema digital, los pequeños emprendimientos y micro-PyMEs enfrentan una **barrera tecnológica y económica** significativa para profesionalizar sus ventas. La mayoría de estos negocios operan de forma fragmentada a través de redes sociales (Instagram, WhatsApp), lo que deriva en los siguientes problemas críticos:

- **Inconsistencia de Inventario:** Al no poseer un sistema centralizado, se producen ventas de productos sin stock real, dañando la reputación del comerciante.
- **Descentralización de Pagos:** La gestión manual de transferencias y el envío de comprobantes por chat genera demoras operativas y riesgos de seguridad.
- **Falta de Identidad Digital:** Los marketplaces genéricos absorben la identidad del vendedor. Existe una necesidad real de una plataforma donde cada negocio mantenga su estética y catálogo de forma independiente.
- **Ausencia de Datos para la Decisión:** El emprendedor no cuenta con métricas sobre qué productos son más visitados o en qué horarios se vende más, impidiendo un crecimiento estratégico.

MarketSaaS surge para resolver esta fragmentación mediante una infraestructura única que democratiza el acceso a herramientas de nivel empresarial (pagos automatizados, chat en vivo y stock transaccional) bajo un costo operativo reducido.

4.2 Alcance y limitaciones del sistema

Alcance:

- **Gestión Multi-tenant:** Capacidad de alojar múltiples tiendas independientes en una misma base de código.
- **Panel Administrativo:** CRUD (Altas, Bajas, Modificaciones) de productos, categorías y configuración de perfil por cada negocio.
- **Módulo de Seguridad:** Registro, login y protección de rutas mediante **JWT** (JSON Web Tokens).
- **Experiencia de Compra:** Catálogo público, carrito de compras persistente y proceso de checkout.
- **Integración Financiera:** Cobro automatizado mediante la API de **Mercado Pago**.
- **Comunicación Real-Time:** Chat bidireccional entre cliente y vendedor vía **SignalR**.
- **Control de Stock:** Validación lógica de existencias antes y después de la transacción.
- **Dashboard de Analítica (Business Intelligence):** Visualización de métricas clave de rendimiento (KPIs), como volumen de ventas, productos de mayor rotación y flujo de caja diario por cada inquilino.

Limitaciones:

- **Logística Física:** El software no gestiona el seguimiento de camiones o flotas de envío propias.
- **Facturación Legal:** No realiza emisión de facturas fiscales (AFIP u organismos similares).
- **Gestión de Devoluciones:** El proceso de logística inversa (devolución de productos) se maneja externamente al sistema.
- **Inteligencia Artificial (IA) y Chatbots:** La plataforma no integra motores de procesamiento de lenguaje natural (NLP) ni respuestas automatizadas por IA, delegando la atención al cliente a la comunicación humana directa.

4.3 Principales requerimientos

Funcionales:

- **Resolución de Multi-tenancy:** El sistema debe identificar dinámicamente al inquilino (negocio) mediante el identificador único en la URL, filtrando automáticamente el catálogo y la configuración estética correspondiente.
- **Gestión de autenticación y Perfiles:** Debe permitir el registro y login de usuarios, diferenciando niveles de acceso (SuperAdmin, Admin de Tienda, Cliente) mediante **Claims** embebidos en el token de sesión.
- **Control de Stock Transaccional:** El sistema debe validar la disponibilidad de existencias antes de permitir el checkout y descontar automáticamente las unidades una vez confirmada la transacción.
- **Gestión de Catálogo Polimórfico:** El administrador debe poder crear, editar y eliminar productos con atributos variables (talle, color, peso) aprovechando la flexibilidad de esquemas de **MongoDB**.
- **Integración de Pasarela de Pagos:** El sistema debe comunicarse con la API de **Mercado Pago**, procesar el cobro y recibir notificaciones asíncronas (**Webhooks**) para actualizar el estado del pedido.
- **Mensajería Síncrona:** Debe proveer un canal de chat en tiempo real para que el cliente realice consultas pre-venta directamente al administrador del comercio.
- **Dashboard de Analítica (BI):** El sistema debe procesar los datos históricos para visualizar métricas de rendimiento: volumen de ventas diario, productos más vendidos y flujo de caja por periodo.

No Funcionales:

- **Seguridad de Datos (JWT):** Toda comunicación con la API debe estar protegida mediante **JSON Web Tokens**. Las contraseñas deben estar cifradas en la base de datos utilizando algoritmos de *hashing* seguros (como **BCrypt**).
- **Escalabilidad Horizontal:** La arquitectura debe estar diseñada para soportar el incremento de inquilinos (tenants) sin requerir cambios estructurales en el código fuente ni degradar el tiempo de respuesta.

- **Persistencia No Relacional:** El gestor de base de datos debe ser **NoSQL (MongoDB)** para garantizar alta disponibilidad y una lectura veloz de documentos JSON complejos.
- **Rendimiento y Reactividad (UX):** La interfaz en **Vue.js** debe cargar los componentes de forma asíncrona, asegurando que las interacciones del usuario (como agregar al carrito) no bloqueen el hilo principal de ejecución.
- **Concurrencia de Datos:** El backend en **.NET** debe manejar peticiones simultáneas de stock mediante programación asíncrona para evitar bloqueos en el servidor.
- **Usabilidad (UX):** La interfaz debe ser intuitiva, con un tiempo de carga de componentes inferior a 2 segundos en condiciones normales de red.
- **Persistencia Polimórfica:** La base de datos debe ser capaz de almacenar productos con diferentes propiedades técnicas sin errores de esquema.
- **Disponibilidad:** El sistema debe estar diseñado para un despliegue en la nube, garantizando un acceso 24/7.

5. DISEÑO DE SOFTWARE

5.1· Arquitectura general

El sistema se basa en una **Arquitectura Desacoplada de N-Capas**, siguiendo el patrón de diseño **API-First**. Esto permite una separación clara entre la interfaz de usuario (Frontend) y la lógica de negocio (Backend), facilitando el mantenimiento y la escalabilidad.

Para resolver la naturaleza **Multi-tenant** del proyecto, se implementa una **Arquitectura de Aislamiento Lógico**:

- **Tenant Resolver:** Un middleware en el backend intercepta cada petición HTTP, extrae el identificador del negocio de la URL y establece el contexto de datos para esa consulta específica.
- **Aislamiento de Datos:** Se utiliza una base de datos única donde cada documento (producto, venta, mensaje) posee un discriminador **TenantId**, garantizando que un comercio nunca acceda a la información de otro.

5.2 Tecnologías a utilizar

5.2.1 Backend

- **Lenguaje y Framework: .NET 8 (C#).** Se selecciona por su alto rendimiento, seguridad tipada y excelente manejo de la programación asíncrona (`async/await`), fundamental para procesar múltiples pedidos en paralelo sin bloquear el servidor.
- **Patrones de Diseño:** Se aplica el patrón **Repository** para abstraer la lógica de acceso a datos y **Dependency Injection (DI)** para mantener un código desacoplado y testeable.

- **Seguridad:** Implementación de **ASP.NET Core Identity** junto con **JWT (JSON Web Tokens)** para una autenticación *stateless*, ideal para aplicaciones distribuidas.

5.2.2 FrontEnd

- **Framework: Vue.js 3.** Elegido por su reactividad y sistema de componentes reutilizables. Permite crear una **SPA (Single Page Application)** que ofrece una experiencia de usuario fluida, similar a una aplicación nativa.
- **Gestión de Estado: Pinia.** Se utiliza para centralizar y proteger el estado global de la aplicación, como la persistencia del carrito de compras y los datos del perfil del usuario autenticado.
- **Navegación: Vue Router** para el manejo de rutas dinámicas que permiten renderizar diferentes tiendas según el parámetro del negocio en la URL.

5.2.3 Gestor de Base de datos

Motor: MongoDB (NoSQL). A diferencia de las bases relacionales (SQL), MongoDB almacena la información en documentos BSON.

5.2.4 Frameworks

- **SignalR:** Framework de Microsoft para **.NET** que facilita la comunicación bidireccional mediante **WebSockets**. Se utiliza para el chat en tiempo real y notificaciones push de ventas.
- **Mercado Pago SDK:** Integración oficial para el procesamiento seguro de pagos, gestión de preferencias y recepción de Webhooks.
- **BCrypt.Net:** Librería para el hashing y salting de contraseñas, asegurando que las credenciales no sean legibles incluso en caso de una filtración de datos.
- **Chart.js / Vue-Chartjs:** Librería para la generación de gráficos dinámicos en el Dashboard de analítica, transformando datos crudos en información visual estratégica.

6. PRUEBAS Y CALIDAD DE SOFTWARE

Para garantizar la robustez de la plataforma **MarketSaaS**, se aplicará un enfoque de pruebas multinivel. El objetivo es asegurar que el aislamiento de datos entre *tenants* sea infranqueable y que las transacciones financieras sean consistentes.

6.1 Estrategia y Niveles de Prueba

Se implementará una estrategia de pruebas piramidal para cubrir desde la unidad mínima de código hasta la experiencia del usuario final:

1. **Pruebas Unitarias:** Realizadas sobre el Backend en **.NET 8** utilizando *xUnit*. Se validarán métodos críticos de forma aislada, como el algoritmo de stock y la generación de Tokens JWT.
2. **Pruebas de Integración:** Verificación de la interoperabilidad entre MongoDB, el SDK de Mercado Pago y SignalR.
3. **Pruebas de Regresión:** Ejecución de pruebas ante cada nueva funcionalidad para asegurar que no se afecten los módulos operativos.
4. **Pruebas de Aceptación de Usuario:** Validación mediante simulaciones de uso real por parte de Administradores de Tienda y Clientes Finales.

6.2 Tabla de Casos de Prueba por Rol de Usuario

A continuación, se detalla la planificación de pruebas específicas diseñadas para validar los permisos y funcionalidades según el nivel de acceso:

Rol	Tipo de Prueba	Descripción del Escenario	Resultado Esperado
Cliente	Integración / Pagos	Iniciar proceso de compra y completar pago en Sandbox de Mercado Pago.	Recibir Webhook, actualizar pedido a "Pagado" y notificar al vendedor.
Cliente	Consistencia de Datos	Intentar comprar una cantidad superior al stock disponible en MongoDB.	El sistema debe bloquear la transacción y mostrar error de disponibilidad.
Admin Tienda	Seguridad (Aislamiento)	Intentar acceder a la configuración de otra tienda mediante la	El Middleware de resolución de Tenant debe denegar el acceso.

		manipulación del ID en la URL.	
Admin Tienda	Tiempo Real (SignalR)	Recibir una consulta de un cliente mediante el chat integrado sin refrescar el navegador.	La notificación y el mensaje deben renderizarse instantáneamente vía WebSockets.
Admin Tienda	Usabilidad	Generar un reporte de ventas mensual desde el Dashboard de analítica.	Los gráficos deben mostrar tendencias de venta precisas desde MongoDB.
SuperAdmin	Funcional	Crear una nueva instancia de negocio y verificar su despliegue inmediato.	El sistema genera el contexto lógico y permite el acceso automático a la nueva ruta/subdominio.

6.3 Pruebas de Calidad No Funcionales

Para elevar el estándar de la tesis, se contemplan validaciones sobre atributos de calidad específicos:

- **Pruebas de Concurrencia:** Se simularán múltiples peticiones de compra simultáneas sobre un mismo ítem de stock para verificar que el manejo de hilos asíncronos en **.NET** evite condiciones de carrera (*race conditions*).
- **Pruebas de Seguridad (Stateless):** Validación de que los **Tokens JWT** caduquen según el tiempo configurado y que no sea posible realizar peticiones una vez cerrada la sesión.
- **Compatibilidad (Responsive):** Verificación de la interfaz en **Vue.js** utilizando herramientas de inspección para asegurar que el Marketplace sea 100% funcional en dispositivos Android e iOS (Mobile-First).

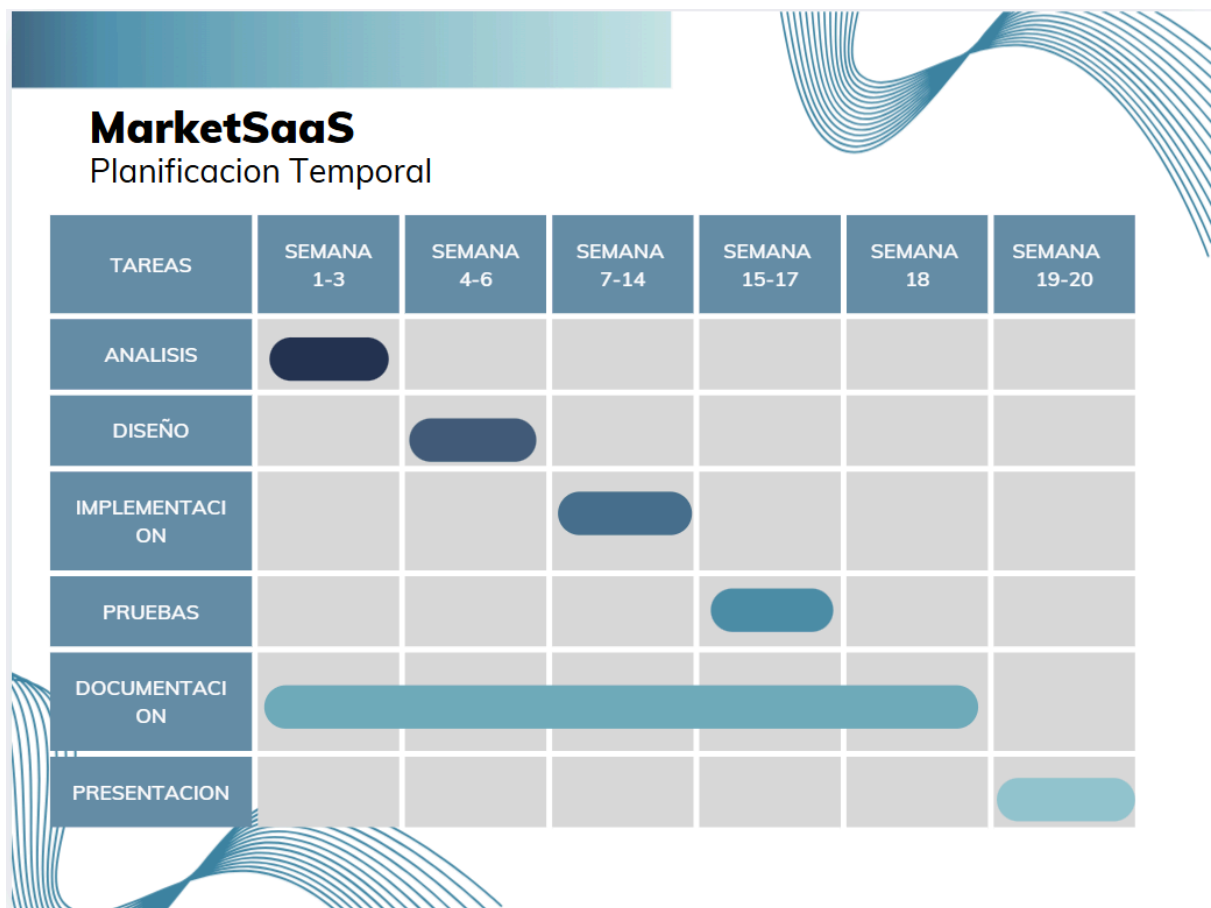
6.4 Estándares de Calidad y Mantenibilidad

El desarrollo se alineará con la norma **ISO/IEC 25010**, enfocándose en:

- **Fiabilidad:** Capacidad del sistema para recuperarse de errores de red durante el procesamiento de pagos.
- **Mantenibilidad:** Uso de patrones de diseño (Repository e Injection) para que el código sea fácil de escalar y corregir por otros desarrolladores.

7. CRONOGRAMA DEL PROYECTO

7.1 Planificación temporal (Diagrama de Gantt)



7.2 Detalle de Etapas Sugeridas

Fase 1: Análisis (Semanas 1 - 3)

- Relevamiento de problemas, necesidades y definición de requerimientos.

Fase 2: Diseño (Semanas 4 - 6)

- Elaboración de esquemas de base de datos (DER) y diseño de Casos de Uso.

Fase 3: Implementación (Semanas 7 - 14)

- Desarrollo de Backend (.NET 8), Frontend (Vue.js) e integraciones (Mercado Pago/SignalR).

Fase 4: Pruebas (Semanas 15 - 17)

- Ejecución de QA integral, pruebas de seguridad y optimización de errores.

Fase 5: Documentación (Semanas 1 - 18)

- *Registro transversal de avances y preparación del video demostrativo final.*

Fase 6: Presentación (Semanas 19 - 20)

- Ajustes finales de escalabilidad y defensa oral ante el tribunal.